

Rapport individuel — Itération 1

Alexis Briend (abd7852) — Projet Long SHDL

13 avril 2026

Contexte et objectif de l'itération

Le projet consiste à développer un simulateur de circuits logiques décrits en langage SHDL, avec une interface JavaFX. L'équipe compte neuf personnes réparties sur six axes (éditeur, parser, automates, simulateur, IHM, intégration). Ma responsabilité porte sur la structure générale de l'application et sur l'**interprétation du SHDL** (ticket SCRUM-23). L'objectif de l'itération 1 était de poser les fondations techniques : arborescence, gestion de dépendances, grammaire SHDL et parser vers un AST exploitable par les modules aval.

Activités réalisées

- **Squelette JavaFX/Gradle** sous `src/main/java/fr/n7/shdl/` (packages `model`, `view`, `controller`), avec `module-info` et `build.gradle`.
- **Grammaire SHDL LL(1)** formalisée dans `parser/111/grammar/` : grammaire déclarative (règles comme objets Java), calcul de FIRST/FOLLOW, vérificateur automatique de conflits LL(1).
- **AST immuable** (`parser/111/ast/`) : hiérarchie `Node` → `Expression/Statement`, champs `final`, collections copiées via `List.copyOf`, non-null garanti, Visitor typé `<R>`.
- **Parseur à descente récursive** avec lookahead 2 sur les instances, messages d'erreur contextuels et enrichis (code + position + attendu).
- **Documentation** : spécification `docs/specs/`, plan d'exécution `docs/plans/`, bilan d'itération `docs/sprint1/`.

Résultats

- **107 tests JUnit verts** répartis sur 23 classes (grammaire, tokens, AST, visiteur par défaut, invariants runtime, cas négatifs, bout-en-bout sur ET, BasculeD, FsmSync, DecBCD).
- **39 fichiers Java** produits sous `parser/111/`.
- Une **limite connue** documente un piège grammatical (`TERM_REST`, wildcard FSM suivi d'un STAR de règle voisine). Un test négatif la rend visible, la correction est reportée à l'itération 2.
- Une revue critique interne a fait émerger cinq défauts (traversal incomplet du `DefaultVisitor`, champs redondants, invariants manquants), tous corrigés dans la branche avant la fin du sprint.

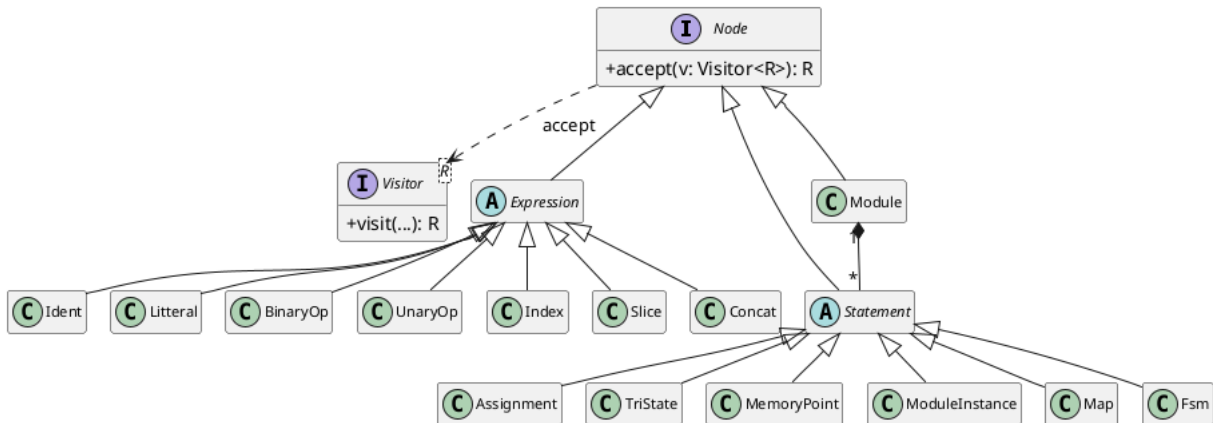


Figure — Diagramme de classes simplifié de l'AST (généré avec PlantUML, source `ast-classes.puml`).

Difficultés et réussites

La principale difficulté a été de garantir que la grammaire reste *réellement* LL(1) après chaque ajout de règle. Le vérificateur de conflits, écrit avant la grammaire, s'est révélé décisif : plusieurs ambiguïtés sur les instances de modules et sur les FSM ont été détectées avant même la première exécution du parser. La réussite principale est l'immuabilité stricte de l'AST, qui rend les passes aval (typage, génération d'automates) triviales à paralléliser et à tester.

Manuel utilisateur

Pré-requis : JDK 21+, JUnit 4 et Hamcrest fournis dans lib/.

Compiler et exécuter les tests du parser :

```
cd parser/ll1
javac -cp ../../lib/junit-4.13.2.jar:../../lib/hamcrest-core-1.3.jar \
      -d ../../bin $(find . -name "*.java") \
      $(find ../../tests/parser/ll1 -name "*.java")
java -cp bin:../../lib/junit-4.13.2.jar:../../lib/hamcrest-core-1.3.jar \
      org.junit.runner.JUnitCore parser.ll1.SanityTest
```

Parser un fichier SHDL depuis le code :

```
Parser p = new Parser(new Lexer(source));
Module m = p.parseFrom(); // retourne l'AST racine
m.accept(monVisiteur);    // passe aval au choix
```

Le code source est publié sur la branche `feature/skeleton-javafx`. Le bilan collectif de fin d'itération se trouve dans `docs/sprint1/2026-04-13-bilan-sprint1.md`.